computers

MDPI

Article

Beyond Snippet Assistance: A Workflow-Centric Framework for End-to-End AI-Driven Code Generation

Vladimir Sonkin [1] and Cătălin Tudose [2,3,*]

[1] Luxoft Serbia, 11079 Beograd, Serbia; vladimirsonkin455@gmail.com
[2] Luxoft Romania, 020335 Bucharest, Romania
[3] Faculty of Automatic Control and Computers, National University of Science and Technology Politehnica Bucharest, 060042 Bucharest, Romania
* Correspondence: catalin.tudose@gmail.com

Abstract: Recent AI-assisted coding tools, such as GitHub Copilot and Cursor, have enhanced developer productivity through real-time snippet suggestions. However, these tools primarily assist with isolated coding tasks and lack a structured approach to automating complex, multi-step software development workflows. This paper introduces a workflow-centric AI framework for end-to-end automation, from requirements gathering to code generation, validation, and integration, while maintaining developer oversight. Key innovations include automatic context discovery, which selects relevant codebase elements to improve LLM accuracy; a structured execution pipeline using Prompt Pipeline Language (PPL) for iterative code refinement; self-healing mechanisms that generate tests, detect errors, trigger rollbacks, and regenerate faulty code; and AI-assisted code merging, which preserves manual modifications while integrating AI-generated updates. These capabilities enable efficient automation of repetitive tasks, enforcement of coding standards, and streamlined development workflows. This approach lays the groundwork for AI-driven development that remains adaptable as LLM models advance, progressively reducing the need for human intervention while ensuring code reliability.

Keywords: artificial intelligence; LLM; AI code review; prompt engineering; routine tasks; software development automation; developer productivity improvement; JAIG

check for updates

Academic Editor: Paolo Bellavista

Received: 29 January 2025
Revised: 20 February 2025
Accepted: 4 March 2025
Published: 6 March 2025

Citation: Sonkin, V.; Tudose, C. Beyond Snippet Assistance: A Workflow-Centric Framework for End-to-End AI-Driven Code Generation. *Computers* **2025**, *14*, 94. https://doi.org/10.3390/computers14030094

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

The rapid evolution of Artificial Intelligence (AI) in software development is reshaping how code is written, tested, and deployed. Tools like GitHub Copilot [1] and Cursor [2] provide real-time context-aware code suggestions, enhancing developer productivity. However, these tools primarily function as assistive mechanisms rather than end-to-end automation solutions. They focus on individual coding tasks, such as auto-completing code snippets or refactoring small code blocks, but lack the ability to orchestrate and automate entire development workflows.

As Large Language Models (LLMs) [3,4] continue to advance, they offer the potential to move beyond isolated snippet assistance and enable full automation of multi-step software development processes. To address this gap, we introduce workflow-centric AI code generation. This is an approach that structures AI-driven development as a chain of prompts, each representing a step in the software engineering process. This framework extends beyond simple code generation by incorporating automated validation using LLMs, test generation, test execution, and iterative refinement, ensuring that AI-generated code not only meets functional requirements but also adheres to correctness and best practices.

By making the workflow iterative, the system continuously refines AI-generated code until it aligns with project specifications and passes validation checks.

1.1. Originality and Novelty

While existing AI-assisted coding tools improve efficiency at the snippet level, they do not automate the entire software development lifecycle (SDLC). Unlike prior solutions that enhance developer productivity through isolated suggestions, our framework offers a structured execution model capable of handling end-to-end development workflows.

The novelty of this approach lies in the following:

- Automating complete software development processes, including code generation, validation, and testing.
- A structured execution pipeline based on the concept of a Chain Of Thought that dynamically refines AI-generated code based on feedback loops, validation mechanisms, and rollback strategies.
- Automatic Context Discovery, dynamically identifying the minimal relevant code files for a given request by analyzing project structure and dependencies. This allows the framework to work efficiently with large projects without feeding the entire codebase, selecting only the necessary code to fulfill the task. It improves LLM accuracy and simplifies the selection of relevant context from extensive codebases.
- Self-healing automation enables the framework to detect and refine incorrectly generated code that does not fulfill the requirements by providing feedback to LLM and automatically regenerating the code.
- Automatic Code Merging, preserving manual code modifications while seamlessly integrating AI-generated updates. The framework compares AI output with manually altered code, allowing developers to modify code alongside AI automation rather than relying solely on automatic code generation.
- Separation of Concerns, ensuring that requirements (what should be done) remain independent from workflows (how it should be done). Developers can modify requirements without changing workflows or update workflows and regenerate the code based on the same requirements.

1.2. Original Contribution

The primary contribution of this work is the development of a workflow-centric framework for AI-driven software automation, which does the following:

- Moves beyond snippet-level assistance to provide comprehensive AI-driven automation.
- Introduces the Prompt Pipeline Language (PPL), a declarative scripting format that allows for the definition of multi-step AI execution workflows.
- Implements automated code validation and rollback mechanisms, ensuring that generated code meets quality standards before integration.
- Demonstrates a reference implementation, JAIG (Java AI-powered Generator), showcasing how structured workflows enhance AI-assisted development.

This approach enables developers to transition from AI-assisted coding to AI-driven automation, significantly reducing manual intervention, improving efficiency, and ensuring software quality, particularly for repetitive tasks.

The remainder of this paper is structured as follows:

- Section 2 (Related Work) reviews existing AI-assisted coding tools and their limitations in automating software development workflows.
- Section 3 (Materials and Methods) introduces the workflow-centric framework, describes the Prompt Pipeline Language (PPL), and presents the reference implementation, JAIG.

- Section 4 (Automating Development with Workflow-Oriented Prompt Pipelines) details the structured execution model, separation of concerns, and AI-assisted code merging.
- Section 5 (Enhancing the Reliability of LLM-Generated Code) explores mechanisms for self-healing automation, automated validation, and error correction.
- Section 6 (Prompt Pipeline Language) explains the iterative execution approach used for workflow automation.
- Section 7 (Practical Applications) demonstrates real-world use cases, including feature implementation, refactoring, debugging, and test generation.
- Section 8 (Results) presents empirical findings on efficiency, accuracy, and reliability.
- Section 9 (Discussion) interprets the results, compares the framework with existing AI-assisted approaches, and outlines limitations.
- Section 10 (Conclusion) summarizes key findings and discusses future research directions.
- Supplementary Materials (Example: Workflow written in Prompt Pipeline Language) provides a detailed example of a PPL-based workflow.

2. Related Work

Previous research and commercial tools, such as GitHub Copilot [1] and Cursor [2], primarily focus on providing immediate assistance by predicting and suggesting code snippets.

GitHub Copilot is an instrument for code completion and automatic programming created by GitHub and OpenAI that integrates with various IDEs, supporting the developer with suggestions for code snippets and auto-completion.

Cursor understands the code, suggests improvements, and can also write pieces of code. It offers AI code completion, error correction, and natural language commands. While it extends its functionality to include features such as in-file navigation and editing, its scope still centers on enhancing individual coding tasks rather than automating comprehensive workflows.

While beneficial, these approaches can overlook larger tasks like the following:

- Refactoring large, complex codebases [5];
- Automating repetitive tasks such as boilerplate code generation for CRUD operations [6–8] or DTO (Data Transfer Object) classes [9];
- Implementing multi-step workflows like creating database models, updating service layers, and generating API endpoints;
- Conducting automated code reviews to ensure adherence to coding standards and best practices;
- Detecting and fixing code smells, such as duplicated code or overly complex methods [5];
- Generating comprehensive unit and integration tests to ensure high code coverage [10];
- Replacing deprecated APIs with modern alternatives;
- Addressing security vulnerabilities by identifying and fixing common issues like SQL injection [11,12] or hardcoded secrets.

GitHub Copilot has already gained popularity among developers, and it may be regarded as a substitute for human pair programming [13]. Sometimes it is referred to as the "AI Pair Programmer". An empirical study that collected and analyzed the data from Stack Overflow and GitHub discussions revealed that the major programming languages used with it are JavaScript and Python, while the main IDE with which it integrates is Visual Studio Code 1.97 [14].

A comprehensive analysis of the results of using GitHub Copilot concludes that the code snippets it suggested have the highest correctness score for Java (57%), while JavaScript is the lowest (27%). The complexity of the code suggestions is low, no matter the programming language for which they were provided. Also, the generated code proved to need simplifications in many cases, or it relied on undefined helper methods [15].

Another metric of large interest is the quality of the code that is generated. According to the assessment from [16], GitHub Copilot was able to generate valid code with a 91.5% success rate. Examining the code correctness, 164 problems were provided, resulting in 47 (28.7%) correctly solved, 84 (51.2%) partially correctly solved, and 33 (20.1%) incorrectly solved.

Also, primarily known for its text generation capabilities, LLM can be adapted for NER (Named-Entity Recognition) tasks and be used for entity recognition in border security [17]. Nature-inspired and bio-intelligent computing paradigms using LLMs have revealed effective and flexible solutions to practical and complex problems [18].

Generating unit tests is also a crucial task in software development, but many developers tend to avoid it, being tedious and time-consuming. It is, therefore, an ideal type of task to be delegated to an AI assistant. An empirical study conducted to generate Python tests with the help of GitHub Copilot [19] assessed the usability of 290 tests generated by GitHub Copilot for 53 sampled tests from open-source projects. The results showed that within an existing test suite, 45.28% of the generated tests were passing, while 54.72% of generated tests were failing, broken, or unusable. If the GitHub Copilot tests are created without an existing test suite in place, 92.45% of the tests are failing, broken, or unusable.

Without a unifying workflow, even with the aid of AI tools, developers must still control every step of the process and perform the integration, testing, refactoring, and merging of generated snippets manually. This reliance on manual oversight and intervention often leads to inconsistencies, errors, and inefficiencies in the development process, limiting the potential productivity gains of AI assistance.

Several attempts were made to automate the entire process of software development, which includes writing, building, testing, executing code, and pushing it to the server. AutoDev [20] is an AI-driven software development framework for autonomous planning and execution of intricate software engineering tasks. It starts with the definition of complex software engineering objectives assigned to AI agents that can perform regular operations on the codebase: file editing, retrieval, building, execution, testing, and pushing.

AlphaCodium [21] follows a test-based, multi-stage, code-oriented iterative process organized as a flow, using LLMs to address the problems of developing code. Its applicability is mainly for regular code generation tasks that cover a large part of the time of a developer.

The influence of AI-driven software development automation redefines traditional practices, offering innovative solutions to long-standing challenges. AI may be applied in different phases of the process, with various results: automated code generation, debugging, maintenance, and decision-making processes [22].

AI is effectively integrated into the business process, pushing various industries, such as software engineering, automation, education, accounting, mining, legal services, and media [23]. Automation was applied to remove roadblocks during the last decades, and AI-based tools may replicate the same mindset nowadays [24].

By adopting standardized workflows, developers can leverage AI more effectively to automate these tasks while ensuring that the generated code aligns with project requirements, coding standards, and best practices, reducing the need for constant manual oversight.

3. Materials and Methods

The methodology consists of several key components as follows:

- Basic Workflow Building Blocks: These are the basic capabilities that enable other, more comprehensive features.
- Prompt Pipeline Language (PPL): A declarative scripting format for defining multi-step automation workflows.
- Reference Implementation (JAIG): A Java-based AI-powered code generation tool that integrates these concepts.

- Evaluation Metrics: Assessing accuracy, efficiency, and reliability of AI-generated code through empirical testing.
- Algorithmic Approach: Automatic Context Discovery, Automatic Rollbacks, and Multi-Step Execution methods.

Each subsection describes the corresponding aspect of the methodology in detail.

3.1. Basic Workflow Building Blocks

To overcome the limitations of existing AI-assisted coding tools, we propose a workflow-centric framework that automates the entire software development lifecycle, from requirements gathering to code generation, testing, and deployment. The framework is designed to address key challenges in AI-driven development workflows and ensure seamless integration of AI-generated outputs into existing systems.

The framework introduces the following foundational components to enhance reliability and automation:

1. Files/Folders Inclusion in Prompts: Enables developers to specify the path of files or folders in a prompt, ensuring that these files are included in the input context.
2. Automated Response Parsing and File Organization: After AI-generated code is produced, it must be structured correctly within the project's directory hierarchy.
3. Automatic Rollbacks: If the generated code fails to meet expectations, the framework reverts changes to maintain a consistent project state.
4. Automatic Context Discovery: Dynamically analyzes project structure and dependencies to determine the minimum set of relevant files required for a given request.

3.2. Implementation of the Prompt Pipeline Language

To facilitate structured execution, we introduce the Prompt Pipeline Language (PPL), a declarative scripting format that allows developers to define multi-step automation workflows. This language enables the execution of chained prompts, where the output of one prompt becomes the input for the next, thereby reducing the need for manual intervention.

The key components of PPL include the following:

- Directives: Commands that control execution flow, such as #repeat-if for retry conditions and #save-to for output file destinations.
- Reusability Mechanisms: Support for placeholders and reusable templates that allow developers to define standardized workflows across multiple projects.
- Self-Healing Mechanisms: Integration of automated test generation and validation to detect errors and trigger corrective actions before merging AI-generated code into production.

PPL is discussed in detail in Section 5.

3.3. Reference Implementation: JAIG (Java AI-Powered Generator)

As a proof of concept, we implemented the proposed framework in JAIG, a command-line tool that automates Java development tasks using AI-driven workflows. JAIG leverages OpenAI's GPT models for code generation and integrates with IntelliJ IDEA for seamless adoption by developers. JAIG features include the following:

1. Automated Code Generation: Developers provide structured prompts containing project context and requirements, which JAIG processes to generate Java classes, methods, and APIs.
2. Workflow Execution: Uses PPL to execute multi-step automation, ensuring that generated code is structured, compiled, and tested before integration.
3. Code Validation and Refinement: Implements automatic rollback mechanisms to revert faulty AI-generated outputs and enables iterative improvement through prompt modifications.

4. Test Generation and Execution: Automatically generates and executes unit tests to ensure the correctness of generated code.
5. Code Merging: Prevents AI-generated code from overwriting manually edited files by performing intelligent diffs and merges.

However, some capabilities (such as Automatic Context Discovery and Automatic Workflow Generation) are still under development.

3.4. Evaluation Methodology

To assess the framework's effectiveness, we conducted a comparative study, tracking developer performance with and without AI-driven automation. The evaluation focused on the following:

- Automation Efficiency: Categorized tasks as fully automated, partially automated, or requiring manual intervention to measure the framework's automation impact.
- Productivity Gains: Analyzed task completion times across code generation, testing, refactoring, and documentation updates.
- Code Reliability: Evaluated compilation success, automated rollbacks, and validation accuracy to ensure AI-generated code correctness.
- Workflow Adaptability: Examined modifications to workflows as an alternative to manual refactoring.

By structuring AI-assisted development into iterative, self-correcting workflows, the study aimed to validate the framework's impact on efficiency, reliability, and scalability in real-world software projects. The results of the measurements following this methodology are presented in Section 8 (Results).

3.5. Algorithmic Approach for Workflow Execution

The core algorithms powering the framework play a critical role in ensuring the accuracy, reliability, and efficiency of AI-driven development workflows. This section outlines the primary algorithms used for Automatic Context Discovery, Automatic Rollbacks, and Multi-Step Prompt Execution.

3.5.1. Automatic Context Discovery Algorithm

As projects grow in complexity, providing the necessary context to LLMs becomes increasingly challenging. In small projects, developers can explicitly specify relevant files and documentation, but in large-scale systems, manually selecting the right context for each AI request is inefficient and error-prone. Without a structured approach, excessive or missing context can lead to irrelevant responses or incomplete code generation.

To address this, the framework introduces an automated context discovery mechanism that dynamically determines the minimum necessary context to resolve a given issue. This is achieved through a two-step process:

Step 1: Generate Project Index

Before an LLM can efficiently assist with development, it must understand the overall structure of the codebase. To enable this, the framework automatically generates a table of contents (ToC) that summarizes the key components of the project, including the following:

- Main classes, interfaces, and their relationships.;
- API endpoints and exposed services;
- Core business logic modules;
- Database schema and key entities;
- Configuration files and external dependencies.

This project index acts as a navigational map, allowing the framework to locate and retrieve relevant components without analyzing the entire codebase.

Step 2: Select Relevant Context

Once the project index is created, the framework dynamically identifies the minimal context required to address the developer's query. This process involves the following:

1. Understanding the Developer's Request: The framework parses the prompt to identify the affected modules, classes, or APIs.
2. Dependency Analysis: The framework traces dependencies between classes and functions to determine what additional files must be included.
3. Automatic Context Assembly: Based on the identified dependencies, the framework dynamically constructs the optimal context before passing it to the LLM.

For example, if a developer requests:

"Modify the user authentication service to support multi-factor authentication".

The framework will:

- Identify the UserService class as the main modification target;
- Retrieve dependencies like UserRepository, AuthController, and SecurityConfig;
- Include relevant API documentation and configuration files;
- Exclude unrelated parts of the project, preventing unnecessary context overload;

3.5.2. Automatic Rollback Handling Algorithm

To maintain stability in automated workflows, the system implements an automatic rollback mechanism whenever AI-generated code needs to be regenerated. Rollback is triggered not only when validation fails but also whenever code regeneration is required, such as updating requirements, modifying workflows, compilation failures, or any other event requiring code regeneration.

The rollback algorithm operates as follows:

1. Snapshot Creation: Before applying AI-generated modifications, the system stores the original code state to ensure recovery if needed.
2. Code Regeneration: When code needs to be updated or re-executed, the system generates new output based on the latest prompt and workflow state.
3. Validation Check: The newly generated code undergoes compilation, static analysis, and automated tests to verify correctness.
4. Rollback Trigger: If validation fails or if regeneration was triggered due to prompt updates, the system automatically restores the last working version and logs the failure for developer review.
5. Loop Prevention: To avoid infinite rollback-regeneration loops, workflows must define a limited number of refinement attempts before escalating the issue for manual intervention.
6. Successful Execution and Code Integration: If validation passes, the AI-generated code is parsed, validated, and merged with the existing codebase.

Automatic rollbacks are essential for maintaining an agile and error-resistant development process. Developers can safely experiment with different prompts, refining them iteratively until the desired output is achieved while ensuring that faulty or incomplete code does not disrupt the stability of the existing codebase, as in Figure 1.

Figure 1. Applying automatic rollback when the generated code is unsatisfactory.

3.5.3. Multi-Step Prompt Pipeline Execution

The Prompt Pipeline Language enables the structured execution of AI-generated workflows. The pipeline execution follows these steps:

1. Step Definition: The workflow is defined using PPL, specifying input dependencies and execution order.
2. Sequential Execution: Each step is processed in order, feeding the output of one prompt into the next.
3. Error Handling and Retries: If a step fails validation, it is retried up to a predefined threshold before triggering a rollback.
4. Final Integration: Once all steps are completed, the final AI-generated output is merged into the project.

By leveraging these algorithms, our framework ensures that AI-driven automation remains robust, scalable, and adaptable to complex software engineering tasks.

3.6. Prompt Directives

To ensure flexibility and configurability in AI-driven automation workflows, the framework provides mechanisms for prompt configuration. These capabilities allow developers to configure how the framework processes the LLM input and post-processes the output.

Prompts can be configured using directives, which define execution behavior, override default settings, and specify storage locations for generated artifacts. Directives override global configuration settings when applied within a prompt; otherwise, the framework defaults to the global configuration.

For example, while the default LLM model might be GPT-4o [25], users may opt for a more powerful GPT-o1 model for specific prompts requiring advanced reasoning capabilities. The GPT-o1 series is designed for complex problem-solving tasks in research, strategy, coding, math, and science domains [26]. By specifying a directive within a prompt, developers can dynamically adjust which model is used.

Some examples of directives include the following:

- #model: <model_name>: selects the AI model used for processing a prompt. Developers can switch between models for different levels of complexity (e.g., GPT-4o for general tasks, GPT-o1 for high-reasoning requirements).
- #temperature: <value>: adjusts the creativity level of responses. A higher value produces more varied outputs, while a lower value ensures deterministic results.
- #save-to: <path_to_file>: saves a generated response to the specified path.

By default, the framework parses LLM responses as Java code and organizes them under /src/main/java following the Maven folder structure [27]. The #save-to directive defines where the generated content should be stored for no-code artifacts, like documentation or configuration files.

By leveraging directives, developers gain fine-grained control over workflow execution, ensuring that AI-driven automation remains adaptable, efficient, and tailored to project requirements.

3.7. AI-Assisted Code Merging

In software engineering, requirements serve as the foundation for product development [28]. In Agile environments [29], the codebase is continuously evolving due to the following:

1. Manual code updates: Developers modify the codebase directly.
2. Automatic code updates: Changes in requirements (prompt modifications) triggering AI-driven code regeneration.

A common challenge in AI-assisted development is managing these concurrent modifications. AI-generated code may overwrite manual updates, leading to lost developer contributions and introducing inconsistencies.

To solve this, the framework introduces AI-assisted code merging, ensuring the following:

- Manual code updates are preserved while incorporating AI-generated updates.
- AI-generated code is adjusted based on recently modified project files.
- Developers can independently modify the codebase or the requirements without introducing conflicts.

Figure 2 illustrates how AI-assisted code merging resolves conflicts between manually updated code and AI-regenerated code due to changes in prompts. The process follows these key steps:

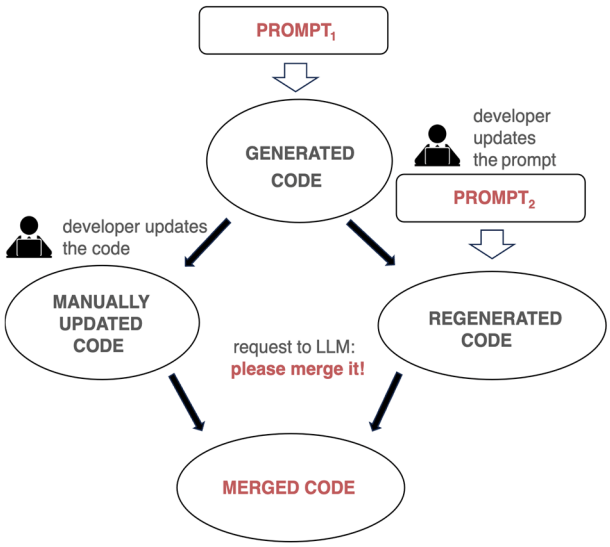

Figure 2. Merging generated output with manually updated code.

1. Initial Code Generation: AI generates code based on an initial prompt (Prompt$_1$).
2. Manual Updates: Developers modify the generated code as needed.
3. Prompt Update and Regeneration: If requirements change, an updated prompt (Prompt$_2$) triggers AI to regenerate the code.
4. Merging Process: Instead of replacing manual updates, the framework requests the LLM to merge the regenerated code with the manually updated version.
5. Final Output: The merged code incorporates both AI-generated updates and developer modifications, ensuring consistency.

This approach prevents code loss, maintains developer control, and allows seamless AI-driven automation with human oversight.

3.8. Reusable Prompt Templates

To enhance efficiency, the framework supports prompt templates, making it possible for prompts to dynamically adapt to project-specific requirements using placeholders, as shown in Figure 3.

- Instead of hardcoding values in every prompt, placeholders such as [[Entity]] and [[requirements]] are used.
- At runtime, these placeholders are automatically substituted with actual values from structured data files (e.g., course.yaml).
- LLM processes the dynamically assembled prompt, ensuring that responses are tailored to specific tasks.

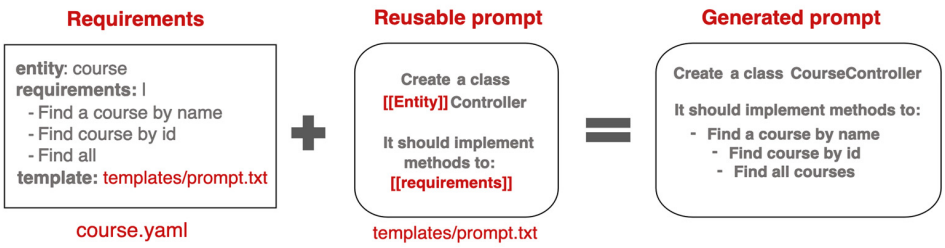

Figure 3. The process of substituting placeholders at runtime.

In Figure 3, placeholders such as [[Entity]] or [[requirements]] are automatically substituted based on provided input, such as course.yaml:

With the foundational components of the framework established, including LLM response parsing, prompt directives, AI-assisted code merging, and reusable templates, we are now ready to create full-scale workflows that implement complex tasks with AI assistance.

4. Automating Development with Workflow-Oriented Prompt Pipelines

Traditional software development follows a structured workflow involving problem analysis, design, coding, testing, and deployment. These phases are typically broken down into smaller steps executed sequentially.

A key principle in LLM-driven automation is the Chain of Thought [30,31], where prompts are processed in sequence, with each output feeding into the next. This approach mirrors the Divide and Conquer strategy [32,33], breaking down complex tasks into manageable, AI-assisted steps.

For example, developing a new feature typically involves the following:

- Updating the domain model;
- Modifying the service layer;
- Creating API endpoints;
- Writing unit tests.

While AI tools assist with individual coding tasks, a prompt pipeline structures the entire workflow, automating multi-step execution, refining outputs iteratively, and minimizing manual intervention. By leveraging structured automation, developers can ensure that generated code remains aligned with requirements, reducing effort while improving software quality.

This approach enables developers to automate the entire feature development process, ensuring that the generated code is consistent with the requirements.

4.1. Prompt Pipeline for Workflow Automation

A prompt pipeline enables structured automation by defining a sequence of prompts, each focusing on a specific development task. This approach ensures consistency and standardization across multiple business domains, simplifying code generation at scale.

Each prompt targets a specific aspect of the development process. The following are examples:

- One prompt generates the domain model;
- Another handles the service layer;
- A separate prompt defines API interactions;
- Additional prompts cover testing and documentation.

By breaking tasks into specialized prompts, the framework ensures higher accuracy, improved modularity, and easier debugging. Additionally, LLMs perform more efficiently and reliably when focused on smaller, well-defined tasks rather than generating large, complex code blocks in a single step. This targeted approach minimizes errors and enhances

predictability, as discussed in more detail in Section 5: Enhancing the Reliability of LLM-Generated Code.

Figure 4 illustrates how a prompt pipeline processes requirements, dynamically inserting them into prompts. The key benefit is that pipelines and requirements remain independent to ensure the following:

- Developers can modify the pipeline to adjust how code is generated;
- Or they can update requirements without altering the workflow.

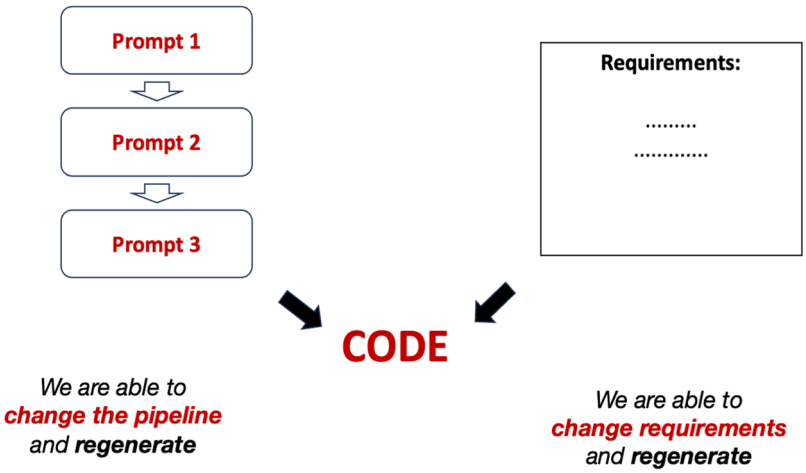

Figure 4. A structured prompt pipeline enables automated code generation, ensuring that requirements and workflow logic remain independent.

This separation of concerns provides greater flexibility and maintainability, making AI-driven development more adaptable to evolving project needs.

4.2. Separation of Concerns in Workflow Automation

Separation of concerns [34] is a key principle of the workflow-oriented approach, ensuring that requirements (what should be done) are decoupled from workflows (how it should be done). This decoupling provides flexibility: the same requirements can be reused across multiple workflows, or workflows can be updated independently without altering the requirements. This modularity enhances maintainability, allows faster iterations, and prevents unintended side effects when making changes.

Additionally, the framework leverages a built-in template engine with support for Velocity Templates, enabling dynamic prompt generation based on requirements. This allows for conditional logic, loops, and reusable placeholders, ensuring that textual prompts can be dynamically structured depending on specific project needs. By combining this templating approach with automated workflows, the system can adapt prompt definitions dynamically, improving both flexibility and efficiency.

Figure 5 illustrates this principle, showing how development workflows and business requirements can be defined independently yet work together to generate consistent, reliable code.

The following two scenarios of independent changes are:

- Changing the workflow: When a workflow is updated (e.g., to generate additional documentation), the same requirements can be reapplied to the updated workflow. Steps that have already been executed (e.g., domain model generation) are not re-executed unless affected by the changes. However, if an early step in the workflow is modified, such as domain model generation, all subsequent dependent steps must be reprocessed to ensure alignment across the entire workflow.

- Changing the requirements: When requirements change, the framework automatically rolls back changes from previous workflows and regenerates only the necessary parts of the codebase, ensuring minimal manual intervention while keeping the implementation aligned with updated specifications. This allows developers to focus on refining the requirements while the system ensures consistency. Ideally, once a workflow is established, developers only need to update the requirements or introduce new ones, while the workflow handles the rest automatically, reducing manual overhead.

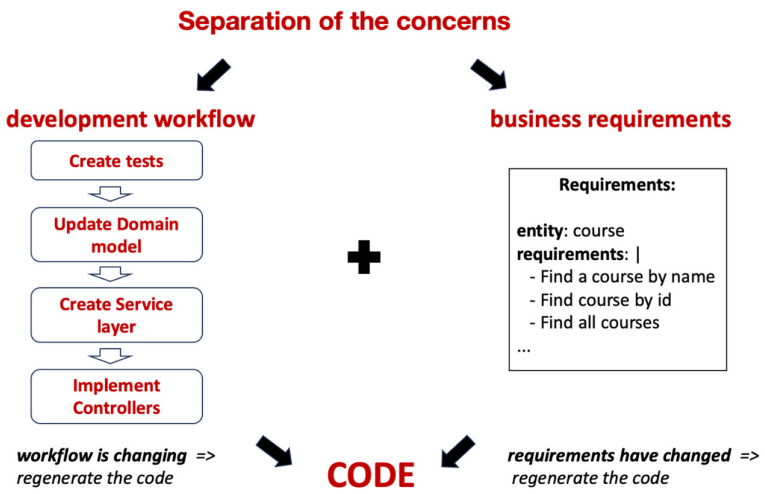

Figure 5. Separation of concerns between development workflow and business requirements.

The principle of separation of concerns is widely applied in software development. For example, the Spring Framework [35] separates configuration from business logic, allowing updates to settings without altering the underlying code. Similarly, in the workflow-oriented AI automation approach, workflows and requirements remain independent, allowing seamless updates to one without disrupting the other.

5. Enhancing the Reliability of LLM-Generated Code

AI-powered code generation offers the potential to streamline development workflows, reducing manual effort in repetitive tasks. However, its effectiveness is heavily dependent on reliability. While LLMs have made significant progress in code generation, they remain prone to misinterpretations, inconsistencies, and errors. Mistakes in one step can propagate through subsequent steps, potentially disrupting the entire workflow. For instance, an incorrectly generated domain model may lead to errors in the service layer and API endpoints, requiring manual intervention to resolve.

To mitigate these risks, breaking down development into smaller, well-defined steps is crucial. LLMs perform best when generating small, focused code snippets rather than entire systems in a single attempt. However, even when using small steps, minor errors can compound over multiple iterations. For example, in a 10-step workflow with a 99% reliability rate per step, the overall success rate drops to 90%, making full automation unreliable without safeguards.

Previously, we introduced the Chain of Thought approach, which structures code generation into sequential steps, ensuring that each step produces validated output before proceeding to the next. This structured execution isolates errors early, preventing flawed code from affecting downstream processes. Instead of regenerating an entire feature due to a single incorrect component, the pipeline identifies and corrects only the affected step.

However, even with structured workflows, errors and inconsistencies can still occur. To address these challenges, this section explores key mechanisms to improve the reliability

of LLM-generated code, focusing on automated validation and self-healing techniques that allow workflows to detect and correct errors efficiently.

5.1. Self-Healing of the Generated Code

Self-healing mechanisms [36] enable the system to recover from errors autonomously, minimizing the need for manual intervention. These mechanisms employ a multi-tiered approach, applying different recovery strategies depending on whether the issue arises from syntax errors, logical inconsistencies, or missing dependencies. Without an automated correction system, developers must manually inspect, debug, and rerun the workflow, defeating the purpose of automation.

To prevent this, self-healing can be implemented through the two following mechanisms:

- Code Validation with Automated Tests: Generates and runs tests to verify that the code aligns with the requirements, identifying errors automatically.
- Code Validation Using LLM Self-Assessment: Uses an LLM to analyze the generated code against predefined requirements, ensuring that even when tests pass, logical errors are caught before they introduce downstream failures.

These mechanisms help maintain code accuracy while reducing the likelihood of manual intervention.

5.2. Code Validation with Automated Tests

Ensuring the reliability of each step in the workflow requires automated validation, which verifies the correctness of the generated code before moving to the next step. Tests should be dynamically generated based on the same requirements, ensuring validation is always aligned with the intended functionality. Figure 6 illustrates this test-based validation workflow.

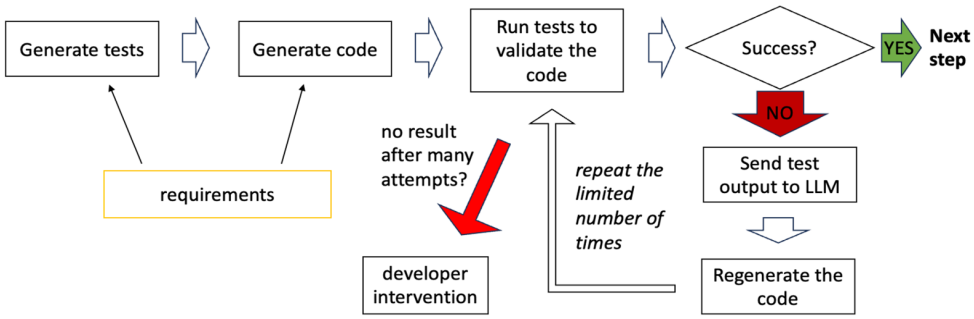

Figure 6. Validating the code with automatically generated tests.

To prevent infinite loops when correcting errors, it is essential to define a retry limit. If the generated code fails validation, it should be retried a predetermined number of times before escalating the issue. This ensures that trivial fixable issues can be resolved automatically while preventing excessive computational overhead.

If the maximum retry attempts are reached without success, the workflow triggers developer intervention, allowing a human to do the following:

- Refine the requirements to improve clarity for LLM processing,
- Select an alternative model for validation or code regeneration,
- Adjust the prompt pipeline to enhance the quality of generated output

Setting an appropriate retry threshold balances automation efficiency with error-handling robustness.

5.3. Code Validation Using LLM Self-Assessment

While automated tests verify syntactic and functional correctness, LLM-based self-assessment ensures that the generated code aligns with higher-level requirements and

intended business logic. A test may pass even if the generated code does not fully align with the requirements. To address this, LLM-based self-assessment introduces an additional layer of validation. The LLM compares the generated code against the original requirements and provides feedback if discrepancies are detected.

This approach ensures that even if all tests pass, the generated code is still reviewed for logical correctness before integration. Figure 7 illustrates this validation process.

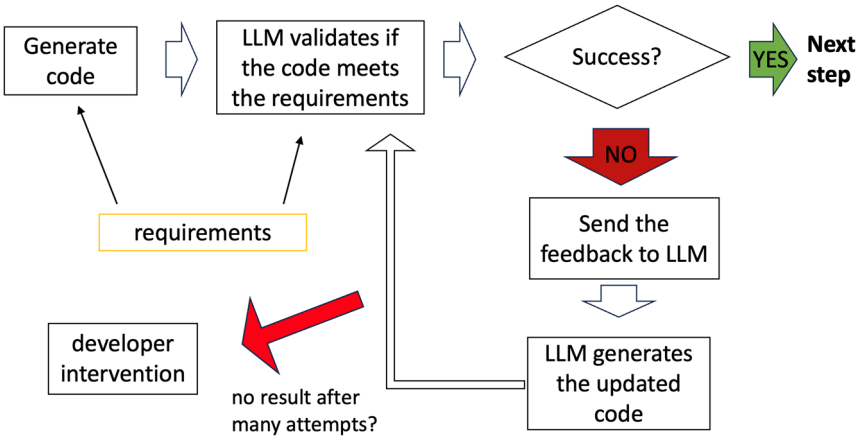

Figure 7. Comparing the code with the requirements and providing feedback.

5.4. Combining Tests and Self-Assessment

Combining automated tests and LLM-based self-assessment offers a comprehensive validation strategy. Tests verify the technical correctness of the code, while self-assessment ensures it meets business requirements, addressing scenarios where tests alone may fall short.

The validating LLM can be different from the generating LLM, adding another layer of reliability as in the following:

- Generating Model: GPT-4o generates the code.
- Validating Model: Claude Sonnet [37,38] validates the code by comparing it to the requirements.

By assigning different models for generation and validation, the system reduces the risk of bias in self-assessment, ensuring that inconsistencies in one model's output can be caught before they introduce downstream failures. This division of responsibilities ensures that even complex, multi-step workflows remain accurate and aligned with expectations.

By integrating automated tests and self-assessment mechanisms, workflows can effectively catch both technical errors and requirement misinterpretations. Combining these strategies with multiple LLMs for generation and validation significantly enhances reliability, reducing the need for manual corrections and improving the feasibility of fully automated code generation.

6. Prompt Pipeline Language

The Prompt Pipeline Language (PPL) provides a structured method for defining and executing AI-driven workflows. It enables prompt chaining, validation, self-healing, and automation, ensuring that generated outputs are reliable, adaptable, and maintainable.

PPL ensures that AI-generated code evolves dynamically, reducing manual intervention while preserving quality. Workflows defined in PPL can be reused, modified, and extended without disrupting previous steps, making automation scalable and efficient.

6.1. Key Features of PPL

PPL enhances AI-driven development by providing the following:

- Prompt Chaining: The output of one prompt automatically becomes the input for the next, structuring complex tasks into smaller, manageable steps.
- Execution Control: Conditions (e.g., validation failures) can trigger retries, alternative strategies, or human intervention.
- Self-Healing and Feedback Loops: If a step fails, the workflow refines itself, using feedback to improve generated results.
- Automated Workflow Updates: Workflows and requirements remain independent, allowing incremental modifications without affecting validated outputs.
- Templated Prompt Execution: PPL supports placeholders and conditional logic, dynamically generating prompts based on structured templates (detailed in Supplementary Materials).

The execution behavior of PPL workflows is governed by directives, which define dependencies, error handling, and automation behavior (see Supplementary Materials for details).

6.2. Iterative Execution Algorithm and Workflow Adaptation

PPL follows an iterative execution model to ensure AI-generated code meets validation criteria before integration. The workflow adapts dynamically, refining outputs through structured validation and self-healing mechanisms.

The process consists of the following steps:

1. Prompt Execution: The prompt is combined with the requirements and sent to the LLM for processing.
2. Validation and Self-Assessment: The generated output is validated using automated tests and self-assessment mechanisms.
3. Self-Healing and Refinement: If validation fails (e.g., due to a compilation error or unmet requirements), targeted feedback is provided, and the prompt is re-executed to improve the response.
4. Human Oversight and Adjustments: If automatic corrections fail after a predefined number of attempts, the system escalates the issue to a developer for manual intervention.
5. Incremental Workflow Updates: When new steps are introduced into the workflow, only the affected steps are re-executed, preserving validated outputs and avoiding unnecessary regeneration.
6. Final Integration and Execution: AI-generated outputs are merged into the existing codebase while maintaining manual modifications. The final code is verified and ready for execution.

This adaptive approach ensures that AI-driven automation remains efficient, scalable, and resilient to changes. A detailed example of a PPL-generated workflow is provided in the Supplementary Materials.

6.3. Automated Workflow Generation

Given that AI is capable of generating code, it is reasonable to explore its potential in generating workflows as well. Manually designing AI-driven workflows can be time-consuming and error-prone, especially for large-scale projects. Automating this process simplifies pipeline creation while ensuring best practices are applied across various use cases.

Workflow Generation Process:

1. Analyze Task Description: The system extracts key requirements from natural language specifications.
2. Define Optimal Prompt Sequence: The generator constructs a logical order of prompts using best practices for validation, execution, and error handling.
3. Apply Execution Directives: Execution conditions, validation rules, and feedback loops are automatically assigned for optimal efficiency.

4. Optimize for Performance: The generator eliminates redundant prompts and ensures efficient dependency resolution.
5. Generate and Validate Workflow: A trial run is performed to verify correctness before deployment.

By automating workflow creation, developers eliminate the need for manual pipeline engineering while improving scalability, adaptability, and maintainability. This approach ensures that AI-driven workflows are not only efficient but also reusable and easily extensible for evolving project requirements.

7. Practical Applications of Workflow-Oriented AI Code Generation

The workflow-oriented approach to AI-driven code generation provides numerous practical applications by automating repetitive and routine tasks that developers frequently encounter. By integrating this framework, developers can optimize various stages of the software development lifecycle, improving efficiency, accuracy, and maintainability. The following applications demonstrate its impact:

7.1. Feature Implementation and Code Generation

- Automated Feature Development: The framework can generate boilerplate code, including REST API endpoints [39], service classes, and database models using reusable prompt pipelines.
- Efficient CRUD Operations: Repetitive tasks such as CRUD generation [6–8], standard controllers, and DTO classes [9] can be fully automated, accelerating development while ensuring consistency.
- Improved Development Workflows: This approach is particularly beneficial in established coding practices where standardized templates allow for seamless automation.

7.2. Code Quality and Optimization

- Automated Code Review: The framework analyzes codebases, detects adherence to coding standards, and flags potential bugs [40]. It generates detailed reports with suggestions for improvement, enabling a more structured and thorough review process.
- Refactoring Large Codebases: The framework enhances maintainability by breaking down large, complex methods into smaller, modular functions, improving readability and efficiency.
- Fixing Code Smells: The system identifies issues such as duplicated code, deep nesting, and long methods [5] and suggests structured refactoring improvements.

7.3. Error Resolution and Refactoring

- Automated Bug Fixing: The framework can detect and resolve simple bugs that do not require extensive debugging. Future iterations may incorporate advanced debugging capabilities.
- Replacing Deprecated APIs: AI-powered automation scans for outdated API usage and replaces them with modern alternatives, provided it has the necessary mappings.
- Security Vulnerability Analysis: The system proactively scans for common security issues, such as SQL injection, and recommends best practices for mitigation.

7.4. Test Generation and Validation

- Automated Unit Test Creation: The framework generates unit tests based on the existing class structure and method signatures, improving test coverage and reducing manual effort.

- Enhanced Test-Driven Development (TDD): Developers can seamlessly integrate AI-generated test cases into their projects, ensuring higher code reliability.
- Automated Test Execution: The system runs tests on generated code to validate correctness, refining and regenerating code if necessary.

7.5. Documentation Generation

- Automated Code Documentation: The framework automatically documents REST API endpoints, class hierarchies, and method details in a structured format, ensuring alignment with the latest code updates. Documentation generation can be integrated into CI/CD pipelines, ensuring documentation remains consistent with ongoing code changes.
- Customizable Output Formats: Documentation can be generated in various formats (e.g., Markdown, HTML, PDF) to accommodate different project needs and improve maintainability.

8. Results

To evaluate the impact of the framework, we conducted a comparative study involving two developers of similar qualification levels. One developer performed tasks manually, while the other used JAIG, the reference implementation of our framework. Both developers logged their time in JIRA, tracking the duration of different tasks, including code review, test generation, API creation, and documentation updates.

The results reveal significant efficiency improvements, especially in repetitive and structured tasks.

8.1. Automation Efficiency

We classified tasks into the following three categories:

- Fully Automated: The task is completed entirely without human intervention. The system generates, validates, and integrates the output based on predefined workflows. The generated results are of sufficient quality to eliminate the need for manual review.
- Partially Automated: The workflow automates most of the process, but a human must review or adjust the output before committing the results.
- Requiring Manual Intervention: The task is too complex or context-dependent to be fully automated. AI may provide recommendations, but a developer must manually implement or finalize the solution.

Structured and repetitive tasks, such as code reviews, test generation, and API creation, were highly automated. Conversely, business logic implementation and complex bug fixes require more human involvement due to contextual decision-making, as illustrated in Table 1.

Table 1. Task automation by category.

Task	Fully Automated	Partially Automated	Manual Intervention
Code Review	80%	20%	0%
Documentation Generation	100%	0%	0%
Unit Test Generation	95%	5%	0%
API Endpoint Creation	90%	10%	0%
Business Logic Implementation	30%	50%	20%
Simple Bug Fixes	20%	50%	30%
Complex Bug Fixes	0%	40%	60%

The framework focuses on automating repetitive development tasks. However, for highly complex, non-repetitive scenarios, conventional AI chat interactions may be more

suitable. As LLM models evolve, the percentage of tasks that require manual intervention is expected to decrease, further expanding automation capabilities.

8.2. Measured Time Savings

Compared to manual workflows, JAIG-enabled automation delivered the following efficiency gains:

- A 92% reduction in code review time, as AI automatically generated structured review reports.
- A 96% decrease in unit test creation time enables rapid test coverage expansion.
- A 100% acceleration in documentation updates, ensuring API and system documentation remained consistent with generated code. This process was fully automated, eliminating the need for manual documentation maintenance.

Although initial workflow creation required time investment, it was compensated by long-term automation benefits, leading to a faster and more structured development process.

8.3. Self-Healing Mechanisms and Reliability

Self-healing mechanisms increase reliability by iteratively refining AI-generated code in response to compilation failures, test failures, or incorrect logic. Instead of requiring manual debugging, the system performs the following:

1. Detects errors and provides feedback to AI.
2. Regenerates code based on the failure context.
3. Repeats until the issue is resolved or escalates to human intervention.

This approach ensures AI-generated code is validated before integration, reducing compilation errors from 8% to 2% and increasing correct implementation rates from 87% to 97%.

Consistency, a key reliability factor, refers to the system's ability to generate stable, reproducible solutions that consistently meet the same functional requirements across multiple executions. JAIG achieved 96% consistency, meaning that while the exact code output may differ, the generated solutions remain functionally equivalent and aligned with the given requirements. This stability minimizes variability in AI-driven development, ensuring predictable and reliable automation.

8.4. Workflow Modification: An Alternative to Code Refactoring

A key advantage of a workflow-driven approach is that modifying workflows replaces the need for manual code refactoring.

Instead of editing existing code, developers can perform either of the following:

1. Adjust workflow rules to change how code is generated.
2. Add new workflow steps to extend functionality, such as automatically updating API documentation whenever a new endpoint is created.

Let us discuss these two opportunities separately.

8.4.1. Modifying Workflow Rules: REST to gRPC Migration

We measured the efficiency of migrating an API from REST to gRPC. In a traditional refactoring approach, developers must manually modify API controllers, rewrite service definitions, update unit tests, and regenerate documentation.

With JAIG and workflow automation, the business requirements remain unchanged, but the workflow is switched from REST API generation to gRPC service generation. The steps related to unit test generation and documentation updates remain intact, ensuring consistency.

As a result, we observed the following:

- Automatic code regeneration without modifying individual files.
- Consistent updates to service definitions, method signatures, and client stubs.
- An 80% faster migration compared to manual refactoring.

8.4.2. Adding New Workflow Steps: OpenAPI Documentation Generation

Another advantage of workflow-driven automation is the ability to seamlessly introduce new steps. We evaluated a case where an API was initially generated without OpenAPI documentation, requiring manual updates afterward.

With workflow automation, we added a single step to automatically generate OpenAPI documentation alongside API creation. This allows the following:

- Eliminated manual effort to keep documentation up to date.
- Ensured consistency between generated API specifications and actual implementations.
- Reduced documentation effort by 90%, as updates were now automated.

By allowing workflows to be adjusted without modifying existing code, this approach provides a more scalable and maintainable alternative to traditional refactoring.

The empirical findings demonstrate that structured workflow automation significantly improves efficiency, reduces manual effort, and enhances code reliability. In the following discussion, we analyze these results in depth, comparing our framework to existing AI-assisted coding approaches, outlining its advantages, and identifying areas for future research.

9. Discussion

9.1. Interpretation of Results

The evaluation confirms that the workflow-centric AI-driven framework boosts efficiency, accuracy, and reliability. Manual coding effort was reduced by 85%, with up to 95% of time savings in testing and refactoring. The 99% compilation success rate after refinements ensures stability, while automated rollbacks and test-driven validation minimize semantic errors and enhance overall code quality.

Based on these results, we identify several key takeaways that highlight the advantages of a workflow-centric AI-driven approach.

The key takeaways are as follows:

1. Highly structured tasks (code reviews, test generation, API creation) are nearly fully automated, leading to significant time savings.
2. Workflows are reusable. The initial setup effort is compensated by long-term efficiency gains.
3. Workflow modifications replace traditional refactoring, making large-scale changes faster and more structured.
4. Self-healing mechanisms improve reliability, reducing error rates and ensuring higher-quality AI-generated code.
5. Incremental automation is possible: new steps (e.g., documentation updates) can be added seamlessly to existing workflows.

9.2. Comparison with Existing AI-Assisted Development Approaches

While AI-powered code assistants such as GitHub Copilot and Cursor provide valuable snippet-based suggestions, they primarily operate as reactive tools, assisting developers in real-time coding. In contrast, our proposed framework automates entire development workflows, covering code validation, refactoring, automated testing, and integration. The inclusion of automatic context, rollback mechanisms, and iterative refinements ensures greater consistency and correctness, distinguishing it from existing AI-assisted development approaches. Unlike Copilot, which may produce contextually incorrect or incomplete

code, the framework continuously evaluates AI-generated solutions, refining outputs based on structured feedback mechanisms, as illustrated in Table 2.

Tools like AutoDev [20] and AlphaCodium [21] represent significant advancements in AI-driven software development but take fundamentally different approaches from our framework. AutoDev focuses on fully autonomous execution of software development tasks within a containerized environment, while AlphaCodium enhances test-based iterative code refinement, particularly for competitive programming. By contrast, our workflow-centric framework integrates AI into developer-guided automation workflows, balancing automation with structured validation, iterative refinement, and human oversight. While each approach has its strengths, a direct comparison is challenging due to differing objectives. A more detailed evaluation of these methodologies is left for future research.

Table 2. Comparison of JAIG vs. GitHub Copilot.

Feature	GitHub Copilot	Workflow-Centric AI Framework (JAIG)
Code Generation	Suggests snippets based on local context	Collects relevant project context, generates structured code, validates outputs, and refines through iterative feedback
Workflow Automation	None, focused on single-task assistance	Full automation of multi-step workflows
Validation and Testing	Requires manual review and testing	Built-in validation, test generation, and self-healing mechanisms
Error Handling	Manual	Automatic rollbacks and iterative refinement
Integration with Human Developers	Complements manual coding	Automates structured workflows while allowing human intervention in complex cases

9.3. Limitations and Challenges

While the workflow-centric AI-driven framework demonstrates significant efficiency gains, some limitations and challenges need to be addressed for broader adoption and further improvement.

9.3.1. Scalability Challenges for Large, Enterprise-Level Codebases

As software projects scale, challenges arise in efficiently handling large, multi-repository codebases with complex interdependencies. The framework's ability to select relevant context dynamically is key, but performance overhead may increase when processing large-scale enterprise systems.

Potential solutions are as follows:

- Optimizing context discovery algorithms to efficiently extract the minimal required code sections for LLMs.
- Implementing incremental code generation to reduce processing overhead by modifying only affected components rather than regenerating entire workflows.
- Exploring the capability of handling large-scale workflows in parallel.

9.3.2. Dependence on LLM-Generated Code and Potential Subtle Errors

LLMs, while powerful, may introduce subtle logic errors, security vulnerabilities, or inconsistencies in generated code, even when passing syntactic and semantic validation. Over-reliance on AI-generated outputs without deeper validation poses risks, particularly in safety-critical applications.

Potential solutions are as follows:

- Enhancing multi-step validation with external static analysis tools and AI-assisted formal verification techniques.

- Leveraging ensemble AI models combining different LLMs or rule-based systems to improve robustness.
- Ensuring a human-in-the-loop review process for critical components before deployment.

9.3.3. Handling Domain-Specific Languages (DSLs)

The framework is primarily designed for general-purpose programming languages, but AI-driven automation faces challenges when handling domain-specific languages (DSLs) with strict syntax and semantics. While the framework already supports generating non-code artifacts (e.g., documentation, configuration files, Cucumber tests, API integration tests) and organizing them via the #save-to directive, DSL-specific constraints may impact the accuracy and usability of AI-generated outputs.

Potential solutions are as follows:

- Extending the framework's prompt pipeline language (PPL) to support custom DSLs and structured metadata.
- Integrating specialized LLMs trained on domain-specific corpora to improve accuracy in DSL-based projects.
- Implementing custom transformation layers that automatically parse LLM-generated responses into valid DSL syntax.

9.3.4. Initial Learning Curve and Adoption Complexity

While the framework provides automation benefits, adopting workflow-based AI development requires the following:

- Familiarity with prompt engineering and structured pipeline-based execution.
- Understanding how to modify workflows to achieve the desired automation outcome.
- Balancing human intervention with AI-driven processes to avoid over-reliance on automation.

Potential solutions:

- Providing predefined workflow templates for common development tasks (e.g., API generation, test automation, refactoring).
- Developing intuitive UI tools to assist in workflow creation and modification, reducing reliance on manual prompt configuration.
- Offering gradual onboarding strategies, allowing teams to transition from semi-automated to fully automated workflows in phases.

9.3.5. Handling Edge Cases and Complex Business Logic

AI-driven workflows excel in structured, repetitive tasks, but challenges arise when dealing with the following:

- Highly domain-specific business logic that lacks clear formalization.
- Scenarios requiring nuanced human decision-making, such as architectural trade-offs or strategic code restructuring.
- Edge cases that involve implicit dependencies across multiple components.

Potential solutions:

- Hybrid AI + Human review approach: Let AI handle structured parts while human developers oversee complex decisions.
- Enriching AI-generated prompts with historical project context to improve decision accuracy for complex scenarios.
- Introducing adaptive self-learning mechanisms, where the framework continuously refines AI behavior based on developer feedback.

9.4. Future Research Directions

To further enhance automation reliability and extend its applicability, future research should explore the following:

1. Dialog-Based Workflow Management: Enabling interactive AI that asks clarifying questions during requirement formulation to ensure all necessary details are provided upfront. This reduces ambiguities and improves the accuracy of AI-generated code.
2. Dynamic Workflow Adaptation: Allowing workflows to adjust in real-time based on project changes, reducing the need for manual intervention and making AI-driven development more responsive.
3. Human-in-the-Loop Automation: Investigating methods for incremental developer feedback throughout the workflow, allowing AI to refine outputs dynamically based on real-time user input rather than only escalating after failure.
4. Self-Optimizing Workflows: Using reinforcement learning and historical data to continuously refine AI-generated workflows, improving efficiency and adaptability over time.
5. Hybrid AI Models for Task Optimization: Combining lightweight AI models for routine tasks with advanced LLMs for complex decision-making, improving efficiency, performance, and cost-effectiveness.
6. Scalability and Deployment in Production: Investigating how workflow-driven automation scales across large projects and multi-developer teams, ensuring maintainability, security, and efficiency in real-world applications.
7. Testing Workflow Reliability: Developing systematic methods to verify AI-generated workflows across varying requirements, ensuring correctness, stability, and consistency.
8. Interactive Workflow Optimization: Studying real-time feedback mechanisms that help developers refine AI-generated workflows dynamically, adapting to evolving requirements and constraints.
9. AI-Assisted Code Explainability and Justification: Research methods to generate human-readable explanations of AI-generated code, providing insights into design choices and ensuring better trust and adoption among developers.
10. AI-Driven Security Analysis and Compliance Enforcement: Investigating how AI can automatically detect security vulnerabilities and ensure compliance with industry regulations (e.g., OWASP, GDPR, HIPAA) before deployment.
11. Cross-Project Knowledge Transfer: Developing AI techniques that learn from past projects to improve workflow recommendations, optimize context selection, and enable intelligent code reuse across multiple repositories.

10. Conclusions

The workflow-centric AI-driven framework advances beyond existing AI-assisted coding tools by automating the entire software development lifecycle. Structured execution, self-healing mechanisms, and automated validation enhance consistency, minimize human intervention, and accelerate software delivery.

As AI technology evolves, this approach has the potential to redefine software engineering by shifting developer focus from routine coding to higher-level architecture and strategic decision-making. Future research will address open challenges such as self-optimizing workflows, adaptive dialog-based workflow management, and enterprise-scale automation. In parallel, near-term improvements will enhance usability, scalability, and real-world adoption.

10.1. Real-World Applications and Industry Adoption

With increasing AI adoption, this framework has the potential to transform enterprise software development by streamlining the following key engineering processes:

- Automated API Development: Generating REST/gRPC services, API documentation, unit, and integration tests with minimal manual effort.
- Continuous Code Review: Integrating into CI/CD pipelines to analyze code changes, enforce coding standards, detect potential issues, and generate structured review reports, ensuring high-quality code before integration.
- AI-Assisted Refactoring: Standardizing modernization efforts by replacing legacy patterns, outdated tools, deprecated libraries, and inefficient code structures with optimized implementations, reducing technical debt and improving maintainability.
- Code Compliance and Security Enforcement: Ensuring generated code adheres to industry standards, security policies, and regulatory requirements (e.g., OWASP, GDPR, HIPAA), reducing security risks and improving auditability.

These capabilities position the framework as a scalable AI-powered solution for software automation, driving efficiency in finance, healthcare, e-commerce, and enterprise SaaS development.

10.2. Future Framework Enhancements

To further refine the framework and expand its applicability, the following areas of improvement are proposed:

- Broader Language Support: Expanding beyond Java to support Python, JavaScript, and C++, enabling adoption across diverse tech stacks.
- Embedding Code Debugging in the Workflow: Integrating AI-driven debugging tools directly into the workflow. After code generation, the debugger will automatically execute the generated code, capturing runtime insights such as variable states, execution flow, and errors. This feedback loop will iteratively refine the code before validation and integration, improving reliability and reducing the need for manual debugging.
- CI/CD Pipeline Integration: Enabling automated deployment of AI-generated code. Enhancements will focus on seamless integration with DevOps tools (e.g., Jenkins, GitHub Actions, GitLab CI/CD) to automate builds, testing, and deployment workflows. Additionally, post-deployment monitoring and rollback mechanisms will enhance reliability and performance.
- Workflow Visualization and Debugging: Introducing graphical representations of workflows for better execution tracking, issue analysis, and optimization. This may include a graphical debugger that maps workflow steps, highlights errors, and enables interactive debugging.
- Integration with Industry-Standard Tools: Strengthening compatibility with IDEs (e.g., IntelliJ, VS Code), testing frameworks (JUnit, Mocha), and project management platforms (Jira, GitLab) to ensure seamless adoption in development environments.

By implementing these improvements, the workflow-centric AI framework can evolve into a fully autonomous software development assistant, enhancing productivity while maintaining software quality and developer control.

Supplementary Materials: The following supporting information can be downloaded at https://www.mdpi.com/article/10.3390/computers14030094/s1, Workflow written in Prompt Pipeline Language Example.

Author Contributions: Conceptualization, V.S.; formal analysis, V.S.; investigation, V.S. and C.T.; methodology, V.S.; software, V.S.; supervision, V.S. and C.T.; validation, V.S. and C.T.; writing—original draft, V.S.; writing—review and editing, C.T. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: Data is available in a publicly accessible repository. https://github.com/sonkin/JAIG (accessed on 1 March 2025) and https://jaig.pro/ (accessed on 1 March 2025).

Conflicts of Interest: Author Vladimir Sonkin is employed by the company Luxoft Serbia. Author Cătălin Tudose is employed by the company Luxoft Romania. The authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. GitHub Copilot. Available online: https://github.com/features/copilot (accessed on 1 March 2025).
2. Cursor, the AI Code Editor. Available online: https://www.cursor.com/ (accessed on 1 March 2025).
3. Iusztin, P.; Labonne, M. *LLM Engineer's Handbook: Master the Art of Engineering Large Language Models from Concept to Production*; Packt Publishing: Birmingham, UK, 2024.
4. Raschka, S. *Build a Large Language Model*; Manning: New York, NY, USA, 2024.
5. Fowler, M. *Refactoring: Improving the Design of Existing Code*, 2nd ed.; Addison-Wesley Professional: Boston, MA, USA, 2018.
6. Bonteanu, A.M.; Tudose, C.; Anghel, A.M. Multi-Platform Performance Analysis for CRUD Operations in Relational Databases from Java Programs using Spring Data JPA. In Proceedings of the 13th International Symposium on Advanced Topics in Electrical Engineering (ATEE), Bucharest, Romania, 23–25 March 2023.
7. Bonteanu, A.M.; Tudose, C.; Anghel, A.M. Performance Analysis for CRUD Operations in Relational Databases from Java Programs Using Hibernate. In Proceedings of the 2023 24th International Conference on Control Systems and Computer Science (CSCS), Bucharest, Romania, 24 May 2023.
8. Bonteanu, A.M.; Tudose, C. Performance Analysis and Improvement for CRUD Operations in Relational Databases from Java Programs Using JPA, Hibernate, Spring Data JPA. *Appl. Sci.* **2024**, *14*, 2743. [CrossRef]
9. Tudose, C. *Java Persistence with Spring Data and Hibernate*; Manning: New York, NY, USA, 2023.
10. Tudose, C. *JUnit in Action*; Manning: New York, NY, USA, 2020.
11. Martin, E. *Mastering SQL Injection: A Comprehensive Guide to Exploiting and Defending Databases*; Independently Published; 2023. Available online: https://www.amazon.co.jp/-/en/Evelyn-Martin/dp/B0CR8V1TKH (accessed on 1 March 2025).
12. Caselli, E.; Galluccio, E.; Lombari, G. *SQL Injection Strategies: Practical Techniques to Secure Old Vulnerabilities Against Modern Attacks*; Packt Publishing: Birmingham, UK, 2020.
13. Imai, S. Is GitHub Copilot a Substitute for Human Pair-programming? In An Empirical Study. In Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings, Pittsburgh, PA, USA, 22–24 May 2022; pp. 319–321.
14. Nguyen, N.; Nadi, S. An Empirical Evaluation of GitHub Copilot's Code Suggestions. In Proceedings of the 2022 Mining Software Repositories Conference, Pittsburgh, PA, USA, 22–24 May 2022; pp. 1–5.
15. Zhang, B.Q.; Liang, P.; Zhou, X.Y.; Ahmad, A.; Waseem, M. Demystifying Practices, Challenges and Expected Features of Using GitHub Copilot. *Int. J. Softw. Eng. Knowl. Eng.* **2023**, *33*, 1653–1672. [CrossRef]
16. Yetistiren, B.; Ozsoy, I.; Tuzun, E. Assessing the Quality of GitHub Copilot's Code Generation. In Proceedings of the 18th International Conference on Predictive Models and Data Analytics in Software Engineering, Singapore, 17 November 2022; pp. 62–71.
17. Suciu, G.; Sachian, M.A.; Bratulescu, R.; Koci, K.; Parangoni, G. Entity Recognition on Border Security. In Proceedings of the 19th International Conference on Availability, Reliability and Security, Vienna, Austria, 30 July–2 August 2024; pp. 1–6.
18. Jiao, L.; Zhao, J.; Wang, C.; Liu, X.; Liu, F.; Li, L.; Shang, R.; Li, Y.; Ma, W.; Yang, S. Nature-Inspired Intelligent Computing: A Comprehensive Survey. *Research* **2024**, *7*, 442. [CrossRef] [PubMed]
19. El Haji, K.; Brandt, C.; Zaidman, A. Using GitHub Copilot for Test Generation in Python: An Empirical Study. In Proceedings of the 2024 IEEE/ACM International Conference on Automation of Software Test, Lisbon, Portugal, 15–16 April 2024; pp. 45–55.
20. Tufano, M.; Agarwal, A.; Jang, J.; Moghaddam, R.Z.; Sundaresan, N. AutoDev: Automated AI-Driven Development. *arXiv* **2024**, arXiv:2403.08299.
21. Ridnik, T.; Kredo, D.; Friedman, I. Code Generation with AlphaCodium: From Prompt Engineering to Flow Engineering. *arXiv* **2024**, arXiv:2401.08500.
22. Alenezi, M.; Akour, M. AI-Driven Innovations in Software Engineering: A Review of Current Practices and Future Directions. *Appl. Sci.* **2025**, *15*, 1344. [CrossRef]
23. Babashahi, L.; Barbosa, C.E.; Lima, Y.; Lyra, A.; Salazar, H.; Argôlo, M.; de Almeida, M.A.; de Souza, J.M. AI in the Workplace: A Systematic Review of Skill Transformation in the Industry. *Adm. Sci.* **2024**, *14*, 127. [CrossRef]
24. Ozkaya, I. The Next Frontier in Software Development: AI-Augmented Software Development Processes. *IEEE Softw.* **2023**, *40*, 4–9. [CrossRef]
25. Chatbot App. Available online: https://chatbotapp.ai (accessed on 1 March 2025).

26. Using OpenAI o1 Models and GPT-4o Models on ChatGPT. Available online: https://help.openai.com/en/articles/9824965-using-openai-o1-models-and-gpt-4o-models-on-chatgpt (accessed on 1 March 2025).
27. Varanasi, B. *Introducing Maven: A Build Tool for Today's Java Developers*; Apress: New York, NY, USA, 2019.
28. Sommerville, I. *Software Engineering*, 10th ed.; Pearson: London, UK, 2015.
29. Anghel, I.I.; Calin, R.S.; Nedelea, M.L.; Stanica, I.C.; Tudose, C.; Boiangiu, C.A. Software Development Methodologies: A Comparative Analysis. *UPB Sci. Bull* **2022**, *83*, 45–58.
30. Ling, Z.; Fang, Y.H.; Li, X.L.; Huang, Z.; Lee, M.; Memisevic, R.; Su, H. Deductive Verification of Chain-of-Thought Reasoning. *Adv. Neural Inf. Process. Syst.* **2023**, *36*, 36407–36433.
31. Li, L.H.; Hessel, J.; Yu, Y.; Ren, X.; Chang, K.W.; Choi, Y. Symbolic Chain-of-Thought Distillation: Small Models Can Also "Think" Step-by-Step. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, Toronto, ON, Canada, 9–14 July 2023; Volume 1, pp. 2665–2679.
32. Cormen, T.H.; Leiserson, C.; Rivest, R.; Stein, C. *Introduction to Algorithms*; MIT Press: Cambridge, MA, USA, 2009.
33. Smith, D.R. Top-Down Synthesis of Divide-and-Conquer Algorithms. *Artif. Intell.* **1985**, *27*, 43–96. [CrossRef]
34. Daga, A.; de Cesare, S.; Lycett, M. Separation of Concerns: Techniques, Issues and Implications. *J. Intell. Syst.* **2006**, *15*, 153–175. [CrossRef]
35. Walls, C. *Spring in Action*; Manning: New York, NY, USA, 2022.
36. Ghosh, D.; Sharman, R.; Rao, H.R.; Upadhyaya, S. Self-healing systems—Survey and synthesis. *Decis. Support Syst.* **2007**, *42*, 2164–2185. [CrossRef]
37. Claude Sonnet Official Website. Available online: https://claude.ai/ (accessed on 1 March 2025).
38. Claude 3.5 Sonnet Announcement. Available online: https://www.anthropic.com/news/claude-3-5-sonnet (accessed on 1 March 2025).
39. Fielding, R.T. Architectural Styles and the Design of Network-Based Software Architectures. Ph.D. Thesis, University of California, Irvine, CA, USA, 2000.
40. Martin, R.C. *Clean Code: A Handbook of Agile Software Craftsmanship*; Pearson: London, UK, 2008.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

Reproduced with permission of copyright owner. Further reproduction prohibited without permission.